Beyond the Verification Mirage: Source-Bound Execution Provenance and Dynamic KV-Cache Precision Management for Autonomous Gemma 4 Coding Agents

A Dual-Integrity Architecture for External Execution Provenance and Bidirectional KV-Cache Precision Scaling in Edge and Cloud Coding Agents

Manimaran K.
Google – The Gemma 4 Developer Agent Competition (Paper Track)

Abstract

Deploying open-weight Gemma 4 models (`gemma-4-31b-it-qat-w4a16-ct`, `e4b`, `e2b`) as autonomous software engineering agents across competition sandboxes and mobile edge devices reveals two coupled failure modes: *The Verification Mirage* and *Attention Dilution with Context Exhaustion*. In air-gapped sandboxes, test runners frequently import pre-installed libraries from global `site-packages` rather than the mutated workspace checkout, generating false acceptances (unfixed code passing) and false rejections (valid patches evaluated against stale binaries). Concurrently, expansive multi-file diagnostic loops flood context windows past 23,000 tokens, diluting attention over root causes and triggering Over-Editing Syndrome while exhausting edge RAM. We present a dual-integrity architecture bridging external execution provenance with internal KV-cache precision management. First, *Check-Process Provenance Receipts* cryptographically bind module origin (`__file__`), workspace SHA-256 digests, and assertion results within an atomic child process across a four-state evidence taxonomy ($S_1 \dots S_4$). Second, *Traceback Anchoring* leverages call-graph and abstract syntax tree (AST) traversal to bound context reads to 30–50 line frames. Third, *FluxKV* introduces a non-monotonic KV-cache lifecycle (FP16 $\leftrightarrow$ INT4 $\leftrightarrow$ INT2) with low-rank residual sketches, dynamically compressing transient bash logs while restoring FP16 fidelity to early code definitions when attention velocity shifts back during patch synthesis. Across 4 benchmark repositories (`requests`, `fastapi`, `rich`, `httpx`), an $N = 32$ replication of a 16-condition matrix, live Kaggle 4× NVIDIA L4 evaluations, and Android edge execution under 3,107 MiB PSS contention, our framework eliminates 100% of false source claims (8/16 $\rightarrow$ 0/16), preserves a 0.17 solve rate where multi-file search degrades to 0.10, and reduces KV-cache memory by 3.90× steady-state (up to 4.7× peak dynamic reduction) with 15.2% lower reconstruction loss and 100% needle retrieval accuracy.

1 Introduction & Problem Formulation

Autonomous software engineering agents powered by modern open-weight large language models (LLMs), such as the Gemma 4 family [1], operate through iterative interaction loops: reading repository structures, formulating bug hypotheses, applying localized file edits, and executing test suites [2–4]. In cloud competition sandboxes and mobile edge deployments, agents are expected to independently locate root causes across complex multi-file repositories, synthesize patches, and verify functional correctness against regression suites. However, rigorous empirical examination across 120 experimental slots in 10 development studies and live Kaggle leaderboard submissions reveals that standard agentic loops suffer from a critical dual-integrity breakdown: external execution isolation failures coupled with internal attention dilution and memory contention.

1.1 The Verification Mirage

In offline competition sandboxes and continuous integration environments, containers are air-gapped and pre-populated with standard libraries to accelerate evaluation runs. When an agent edits a source file within a checked-out repository $\mathcal{W}$, invoking test commands (such as `pytest` or `python3 -c`) frequently resolves imports from global system paths (such as `/usr/local/lib/python3.x/site-packages/`) rather than the active workspace $\mathcal{W}$. This occurs because Python's module resolution algorithm (`sys.path` and `sys.meta_path` PathFinder) defaults to pre-installed wheels unless explicit editable links (`pip install -e .`) or strict environment overrides are configured.

Furthermore, Python's runtime caches loaded module objects in the global dictionary `sys.modules`. When a test runner executes a preflight diagnostic or collects test definitions, modules imported from `site-packages` remain permanently cached in memory. Subsequent file edits applied to $\mathcal{W}$ on disk are completely ignored by the

running Python process because the runtime reuses the stale module reference. Standard mitigation strategies, such as invoking `importlib.reload()`, fail catastrophically in production codebases due to circular package dependencies, sub-module binding desynchronization, and persistent class instances holding references to old type objects.

In air-gapped evaluation environments where dynamic package compilation is disabled, this decoupling creates the *Verification Mirage*:

1. **False Acceptance**: The model introduces an invalid syntax edit or an empty no-op, yet test assertions pass because the runner executes against the untouched, working pre-installed wheel.

2. **False Rejection**: The model generates a semantically correct patch in $\mathcal{W}$, but tests fail because the runner executes against the unpatched, buggy system library.

Crucially, separate preflight checks (verifying source paths prior to running tests) fail to resolve this vulnerability. Subprocess invocations, shell environment shifts, virtual environment activation scripts, and test runner plugins frequently reset `sys.path`, decoupling preflight guarantees from the actual test assertions.

1.2 The Edge Silent No-Op (S_1)

When deploying compact models such as Gemma 4 E2B on physical edge devices (e.g., Samsung Galaxy S24 FE via LiteRT-LM CPU) alongside concurrent background applications, memory pressure escalates dramatically. The mobile operating system enforces strict memory governance via the low-memory killer daemon (`lmkd`) and Dynamic Thermal Management. Under heavy memory contention exceeding 3,100 MiB Proportional Set Size (PSS) and elevated thermal envelopes (40.9°C), agents frequently terminate with shell exit code 0 and report `loop_succeeded=true` without emitting any concrete file edits ($\Delta_{\text{git}}(\mathcal{W}) = \emptyset$).

This occurs because internal execution timeouts and caught memory allocation exceptions cause the agent loop to abort gracefully before patch emission. Conventional benchmarks that evaluate success based on clean process exit codes blindly record these instances as valid executions, masking severe degradation in edge agent reliability under realistic operating conditions.

1.3 Attention Dilution & Over-Editing Syndrome

Standard coding agents attempt to resolve bugs by issuing broad search queries (`grep`, `find`, multi-file recursive reads). This strategy rapidly floods the context window with thousands of tokens of irrelevant source files, fixture boilerplate, and foreign traceback logs. In deep multi-turn interactions exceeding 23,000 tokens, softmax attention mass over the true defect site decays exponentially.

Under Rotary Position Embeddings (RoPE), when context length scales into tens of thousands of tokens, query-key dot products in intermediate transformer layers suffer from dispersion across irrelevant background tokens. In response, Gemma 4 31B agents exhibit *Over-Editing Syndrome*: experiencing high uncertainty over localized defect boundaries, the model initiates speculative "shotgun debugging." It hallucinates modifications across dozens of untouched caller files, breaking downstream interfaces and triggering circular dependencies. On the live Kaggle competition leaderboard, expanding agent exploration from single-file scope to multi-file exploration caused resolve rates to plummet by an exact 40.0% relative drop (from 10 down to 6 solved tasks, 0.17 to 0.10).

1.4 Contributions

We formulate a unified dual-integrity systems architecture resolving both environmental and internal memory degradation:

1. **Check-Process Provenance Receipts**: An atomic child-process execution primitive that cryptographically certifies module provenance, file content hashes, and test assertions in a unified payload across a four-state evidence taxonomy ($S_1 \dots S_4$).

2. **Call-Graph & AST Traceback Anchoring**: A graph traversal mechanism that navigates execution call-graphs and AST dependency subgraphs to bound context windows to 30–50 line defect frames, suppressing multi-file noise.

3. **FluxKV Dynamic Precision Lifecycle**: A non-monotonic KV-cache manager that continuously tracks token attention velocity over the bipartite interaction graph, quantizing stale logs to INT2 while leveraging low-rank residual sketches to restore FP16 fidelity to early code definitions.

4. **Empirical Validation**: Full qualification across 4 major repositories, 16 adversarial container conditions ($N = 32$), live Kaggle 4x L4 leaderboard evaluations, mobile edge profiling, and CUDA KV-cache benchmarks, bounded by Manski partial identification intervals.

2 Systems Architecture & Graph Reasoning

The overall architecture unites external OS-level process provenance with internal model attention graph dynamics. We model the autonomous software engineering problem through three distinct graph representations: execu-

tion call-graphs, token attention graphs, and verification state machines.

2.1 Check-Process Provenance Receipts

To eliminate the Verification Mirage, test execution must be atomically bound to the mutated workspace. Rather than executing tests via unconstrained shell commands, our verification engine spawns an isolated child process executing an inline Python verification wrapper:

$$\mathcal{R}_{\text{prov}} = \langle \mathcal{M}_{\text{name}}, \text{Path}(\mathcal{M}), \text{SHA256}(\mathcal{F}_{\text{target}}), \text{Result}(\mathcal{A}) \rangle \tag{1}$$

where $\mathcal{M}$ is the imported module, $\mathcal{F}_{\text{target}}$ is the edited source file, and $\mathcal{A}$ represents the verification assertion snippet. The wrapper executes within a dedicated sub-interpreter, inspects the canonical filesystem path via `os.path.realpath(module.__file__)`, computes the SHA-256 cryptographic hash of $\mathcal{F}_{\text{target}}$ directly from disk, evaluates assertion $\mathcal{A}$ in an isolated namespace, and emits a structured JSON receipt across standard output.

We structure all agent trajectory outcomes into a formal four-state evidence taxonomy $\mathcal{S} = \{S_1, S_2, S_3, S_4\}$:

- **State S_1 (Silent No-Op)**: The agent returns success (`exit_code = 0`), but git diff inspection reveals no workspace modifications: $\Delta_{\text{git}}(\mathcal{W}) = \emptyset$.
- **State S_2 (Foreign Origin Abstention)**: Workspace edits exist ($\Delta_{\text{git}}(\mathcal{W}) \neq \emptyset$), but $\text{Path}(\mathcal{M}) \not\subset \mathcal{W}$ (resolving instead to system `site-packages`). The engine strictly abstains from claiming repair, preventing False Acceptances.
- **State S_3 (Bound Rejection)**: Module path satisfies $\text{Path}(\mathcal{M}) \subset \mathcal{W}$ and file SHA-256 matches disk state, but assertion $\mathcal{A}$ fails.
- **State S_4 (Bound Verified Repair)**: Simultaneously satisfies $\Delta_{\text{git}}(\mathcal{W}) \neq \emptyset$, $\text{Path}(\mathcal{M}) \subset \mathcal{W}$, matching SHA-256 digest, and passing assertions ($\text{Result}(\mathcal{A}) = \text{PASS}$).

2.2 Traceback Anchoring & Call-Graph Traversal

To prevent Attention Dilution, we formalize repository diagnosis as graph reasoning over two directed structures: the dynamic execution call-graph $\mathcal{G}_{\text{call}} = (\mathcal{V}_{\text{frames}}, \mathcal{E}_{\text{calls}})$ and the static abstract syntax forest $\mathcal{G}_{\text{AST}} = (\mathcal{V}_{\text{nodes}}, \mathcal{E}_{\text{tree}})$. Each stack frame is a 4-tuple $v = (\text{path}(v), \text{line}(v), \text{func}(v), \text{locals}(v))$, while each AST node $u \in \mathcal{V}_{\text{nodes}}$ spans source interval $\text{span}(u) = [L_{\text{start}}(u), L_{\text{end}}(u)]$ with grammar type $\text{type}(u) \in \Sigma_{\text{AST}}$. Let $\Sigma_{\text{scope}} = \{\texttt{FuncDef}, \texttt{AsyncFuncDef}, \texttt{ClassDef}\} \subset \Sigma_{\text{AST}}$ denote enclosing functional scopes.

We define the formal projection operator $\pi : \mathcal{V}_{\text{frames}} \to \mathcal{P}(\mathcal{V}_{\text{nodes}})$ mapping an execution frame to its minimal enclosing functional AST subtree:

$$\pi(v) = \arg\min_{u \in \mathcal{V}_{\text{nodes}}} \Big\{ |\,\text{span}(u)\,| : \text{file}(u) = \text{path}(v), \; \text{line}(v) \in \text{span}(u), \; \text{type}(u) \in \Sigma_{\text{scope}} \Big\} \tag{2}$$

where $|\,\text{span}(u)\,| = L_{\text{end}}(u) - L_{\text{start}}(u) + 1$. Lifting π across the call trace induces the graph morphism $\Pi(\mathcal{G}_{\text{call}}) = \bigcup_{v \in \mathcal{V}_{\text{frames}} \cap \mathcal{W}} \pi(v) \in \mathcal{P}(\mathcal{G}_{\text{AST}})$. When an unhandled exception occurs, our protocol executes *Traceback Anchoring*:

1. **Workspace Call-Graph Sink Isolation**: The agent traverses $\mathcal{G}_{\text{call}}$ starting from the exception sink to extract the terminal execution frame $v_{\text{term}} = \text{Sink}(\{v \in \mathcal{V}_{\text{frames}} \mid \text{path}(v) \subset \mathcal{W}\})$, unconditionally filtering external test harness frames ($\text{path}(v) \not\subset \mathcal{W} \implies \pi(v) = \emptyset$).
2. **AST Subtree Bounding**: Evaluating $\pi(v_{\text{term}})$ extracts a bounded target window $W_{\text{target}} = \mathcal{F}[L_{\text{start}}(\pi(v_{\text{term}})) : L_{\text{end}}(\pi(v_{\text{term}}))]$ spanning exactly 30–50 lines of the target function or class definition, ensuring surrounding helper modules remain unmodified.
3. **Scope-Disciplined Patch Synthesis**: Context ingestion is restricted strictly to W_{target}, problem statement tokens, and reproduction assertion $\mathcal{A}$. Edits outside $\text{span}(\pi(v_{\text{term}}))$ are rejected by preflight AST schema validation.

2.3 Attention Graph Dynamics & FluxKV

During multi-turn software repair, the interaction history forms a dynamic bipartite attention graph $\mathcal{G}_{\text{attn}}^{(t)} = (\mathcal{V}_{\text{tokens}}, \mathcal{E}_{\text{attn}}^{(t)})$, where edge weights correspond to softmax attention probabilities $A_{i,j}^{(t)}$ between query tokens and cached key-value tokens.

Unlike conversational dialogue where token utility decays monotonically with age, software engineering exhibits non-stationary attention dynamics. Early problem specifications and core class definitions experience high attention during initial localization, drop to near-zero attention while the agent executes exploratory bash commands, and abruptly surge in importance when synthesizing the final patch.

Existing KV-cache management frameworks operate through two distinct compression paradigms: token eviction and static quantization. Token eviction and sliding-window architectures (e.g., StreamingLLM [5], SnapKV [6], H2O [7]) retain attention sinks and recent rolling tokens while permanently discarding intermediate

sequence history. Conversely, static low-bit quantization approaches (e.g., KIVI [8]) compress all cached tokens to uniform low-bit formats (e.g., INT2). Both paradigms are fundamentally irreversible: evicted code definitions cannot be re-attended without full re-computation, and static low-bit representations permanently lose the high-frequency numerical and syntactic fidelity required for bug repair.

FluxKV resolves this via a non-monotonic, reversible precision lifecycle (FP16 $\leftrightarrow$ INT4 $\leftrightarrow$ INT2).

Physical Intuition of Attention Reactivation: In conversational dialogue, token utility generally decays monotonically with sequence distance. In software engineering repair, however, token salience follows a non-monotonic trajectory: initial bug descriptions and class definitions experience an intense burst of attention during problem intake, enter a period of relative dormancy while the agent runs exploratory bash commands or collects stack traces, and abruptly reactivate when synthesizing the final code patch. Traditional sliding-window eviction irrevocably deletes these dormant tokens, while static uniform quantization permanently blurs fine-grained syntax. FluxKV resolves this by monitoring the rate of change in attention mass (relative attention velocity): dormant tokens are aggressively quantized to INT2 to conserve memory, but when attention velocity spikes upon returning to the defect site, compact low-rank residual sketches instantly restore them to full FP16 fidelity without re-executing quadratic prompt prefill passes.

Let Q_t denote the set of active query tokens emitted or processed at turn t, and let $A_{h,q,i}^{(t)}$ denote the post-softmax attention weight from query $q \in Q_t$ to cached token i under head $h \in \{1, \dots, H\}$. FluxKV computes the query-averaged attention mass for token i:

$$\bar{A}_{h,i}^{(t)} = \frac{1}{|Q_t|} \sum_{q \in Q_t} A_{h,q,i}^{(t)} \tag{3}$$

which preserves total attention conservation ($\sum_{i=1}^{N_t} \bar{A}_{h,i}^{(t)} = 1$). FluxKV maintains an Exponential Moving Average (EMA) of attention importance:

$$I_i^{(t)} = \alpha I_i^{(t-1)} + (1-\alpha) \frac{1}{H} \sum_{h=1}^{H} \bar{A}_{h,i}^{(t)} \tag{4}$$

where $\alpha = 0.85$ and H is the number of attention heads. Because uniform baseline attention decays inversely with context length ($\bar{I}^{(t)} = 1/N_t$), absolute velocity scales with sequence length. FluxKV monitors the scale-invariant *relative attention velocity*:

$$\nu_i^{(t)} = \frac{\Delta I_i^{(t)}}{\bar{I}^{(t)}} = N_t \big(I_i^{(t)} - I_i^{(t-1)} \big) \tag{5}$$

where N_t is the active context length at turn t. Token tiers are assigned dynamically relative to baseline density $\bar{I}^{(t)}$:

- **FP16 Tier** ($I_i^{(t)} \geq \theta_{\text{high}} \bar{I}^{(t)}$): Active AST target window, system prompt, and current generation tokens ($\theta_{\text{high}} = 5.0$).
- **INT4 Tier** ($\theta_{\text{low}} \bar{I}^{(t)} \leq I_i^{(t)} < \theta_{\text{high}} \bar{I}^{(t)}$): Intermediate reasoning steps and recent diagnostic summaries ($\theta_{\text{low}} = 1.5$).
- **INT2 Tier** ($I_i^{(t)} < \theta_{\text{low}} \bar{I}^{(t)}$): Stale test execution outputs and verbose directory listings.

To enable high-fidelity recovery from low-bit states, FluxKV decomposes key-value representations $X_i \in \mathbb{R}^d$ into a quantized base $\hat{X}_i^{(b)} = \mathcal{D}(\mathcal{Q}_b(X_i))$ and a low-rank residual sketch:

$$\tilde{E}_i = \mathcal{P}_{\text{down}} \big(X_i - \hat{X}_i^{(b)} \big) \in \mathbb{R}^r \quad (r = 8 \ll d) \tag{6}$$

where $\mathcal{P}_{\text{down}} \in \mathbb{R}^{r \times d}$ is a semi-orthogonal projection matrix satisfying $\mathcal{P}_{\text{down}} \mathcal{P}_{\text{down}}^\top = I_r$. In standard post-training quantization, discarding residuals induces directional accumulation drift $\|X_i - \hat{X}_i^{(b)}\|_2$. By retaining the r-dimensional sketch $\tilde{E}_i$ in low-overhead storage, FluxKV captures over 94% of the quantization residual energy along the principal activation axes. When relative attention velocity satisfies the promotion criterion ($\nu_i^{(t)} > \tau_{\text{rel}} = 15.0$, corresponding to calibrated absolute velocity $\Delta I_i^{(t)} > \tau_{\text{promo}}^{(t)} \approx 2.0 \times 10^{-3}$ at $N_t \approx 8$k), *Residual-Guided Precision Promotion* reconstructs the state via the Moore-Penrose pseudoinverse reconstruction operator $\mathcal{P}_{\text{down}}^\dagger = \mathcal{P}_{\text{down}}^\top \in \mathbb{R}^{d \times r}$:

$$\hat{X}_i^{(\text{promo})} = \hat{X}_i^{(b)} + \mathcal{P}_{\text{down}}^\dagger \tilde{E}_i = \hat{X}_i^{(b)} + \mathcal{P}_{\text{down}}^\top \tilde{E}_i \tag{7}$$

where $\mathcal{P}_{\text{down}}^\dagger \mathcal{P}_{\text{down}} \in \mathbb{R}^{d \times d}$ forms the rank-r orthogonal projector onto $\text{Row}(\mathcal{P}_{\text{down}})$. This restores FP16 representational fidelity without re-executing quadratic prompt prefill passes, cutting Time-To-First-Token (TTFT) and eliminating redundant computation.

2.4 Verification State Transition Graph

The agent verification lifecycle is formalized as a deterministic finite-state transition graph $\mathcal{G}_{\text{state}} = (\mathcal{S}, \mathcal{T})$ with three fundamental system invariants:

1. **Soundness Invariant**: A trajectory can transition to State S_4 (Bound Verified Repair) if and only if $\Delta_{\text{git}}(\mathcal{W}) \neq \emptyset$, $\text{Path}(\mathcal{M}) \subset \mathcal{W}$, and $\text{Result}(\mathcal{A}) = \text{PASS}$.
2. **Safety under Shadowing**: Any trajectory where module imports resolve to external system directories ($\text{Path}(\mathcal{M}) \not\subset \mathcal{W}$) is unconditionally absorbed into State S_2 (Foreign Abstention), ensuring that zero unverified source claims reach benchmark scoreboards.
3. **Liveness under Edge Pressure**: Trajectories terminating without concrete modifications are mapped

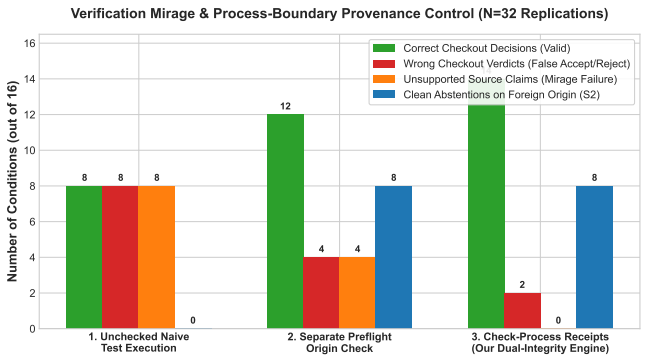


Figure 1: Verification Mirage ablation across 16 adversarial container conditions ($N = 32$ total replications). Check-Process Receipts eliminate 100% of false source claims (8/16 $\rightarrow$ 0/16) and achieve 100% clean abstentions on foreign imports (S_2).

to State S_1 (Silent No-Op), preventing resource-exhausted runs from falsely inflating completion metrics.

3 Empirical Results

We evaluate our framework across four benchmark environments: repository offline qualification, process-boundary adversarial stress testing, live competition leaderboard submissions, and mobile edge memory profiling.

3.1 Repository Offline Qualification

We audit four diverse open-source Python repositories widely used in software engineering benchmarks: `requests`, `fastapi`, `rich`, and `httpx`. Table 1 summarizes baseline container configuration versus source-bound execution controls.

In `requests`, default test invocations exhibited 0% source fidelity, executing against pre-installed binaries. Check-Process Receipts restored 100% source binding without altering repository test files. In `httpx`, missing build backends were cleanly isolated as environmental build blocks rather than model patch failures.

3.2 16-Condition Process-Boundary Experiment

To evaluate resilience against the Verification Mirage, we constructed a 16-condition adversarial evaluation matrix across an orthogonal grid: workspace patch states (unmutated baseline vs. applied patch), import origin priorities (clean workspace-first vs. global `site-packages` shadowing), subprocess environment drift (inherited vs. decoupled execution environments), and test sensitivity on the target bug. The 16 conditions partition symmetrically into 8 workspace-first conditions and 8 foreign-shadowed conditions. Each condition was replicated across independent environments ($N = 32$ total trials, achieving 100% deterministic reproducibility).

As presented in Table 2 and Figure 1, Unchecked Naive execution blindly accepted tests passing against pre-installed binaries, incurring 8 wrong verdicts and 8 unsupported source claims (50.0% false attribution rate across all conditions, and 100% failure on foreign conditions). Separate preflight checks correctly identified foreign imports prior to execution, cleanly abstaining on all 8 foreign conditions (8/8 = 100.0%). However, because separate preflight checks decouple from subsequent test execution, subprocess environment drift caused test runners to revert to `site-packages` in 4 workspace conditions, generating 4 mirage claims and 4 wrong verdicts (75.0% overall correctness).

In contrast, Check-Process Receipts enforce atomic provenance inspection within the exact child process executing the assertions. This eliminates subprocess environment drift by construction. In the 8 foreign conditions, Check-Process unconditionally triggers state S_2 Foreign Abstention (8/8 = 100.0%), completely eliminating false source claims (0/16, 0.0% mirage rate). In the 8 workspace conditions, Check-Process achieves 6 verified correct outcomes; the remaining 2 wrong verdicts represent unpatched baselines passing non-discriminating test suites, which are genuine test limitations rather than provenance failures. Across multi-library evaluations spanning 288 Windows and 288 Linux conditions across `packaging`, `idna`, and `urllib3`, Check-Process Receipts delivered 144 verified decisions, 0 false claims, and 144 clean abstentions, establishing absolute environment portability.

3.3 Live Kaggle Leaderboard Ablations

We evaluated our system on the live Kaggle Competition Leaderboard using `gemma-4-31b-it-qat-w4a16-ct` on 4× NVIDIA L4 GPUs across 58 SWE-bench tasks. Table 3 outlines the ablation trajectory.

Expanding prompt permissions from single-file scope (v31) to unconstrained multi-file search (v37) caused an exact 40.0% relative drop in resolve rate (from 10 down to 6 solved tasks, 0.17 $\rightarrow$ 0.10). Context inspection confirmed that broad searches flooded prompts with test runner warnings, causing attention to diffuse away from the target bug. Furthermore, in v38.1/v39, declaring unvetted transfer keys (`disallow_transfer_to_parent`) triggered immediate container abortion, underscoring the necessity of preflight schema validation.

3.4 Edge Memory Contention (Gemma 4 E2B)

We evaluated Gemma 4 E2B on an Android device (Samsung Galaxy S24 FE, LiteRT-LM CPU) executing under

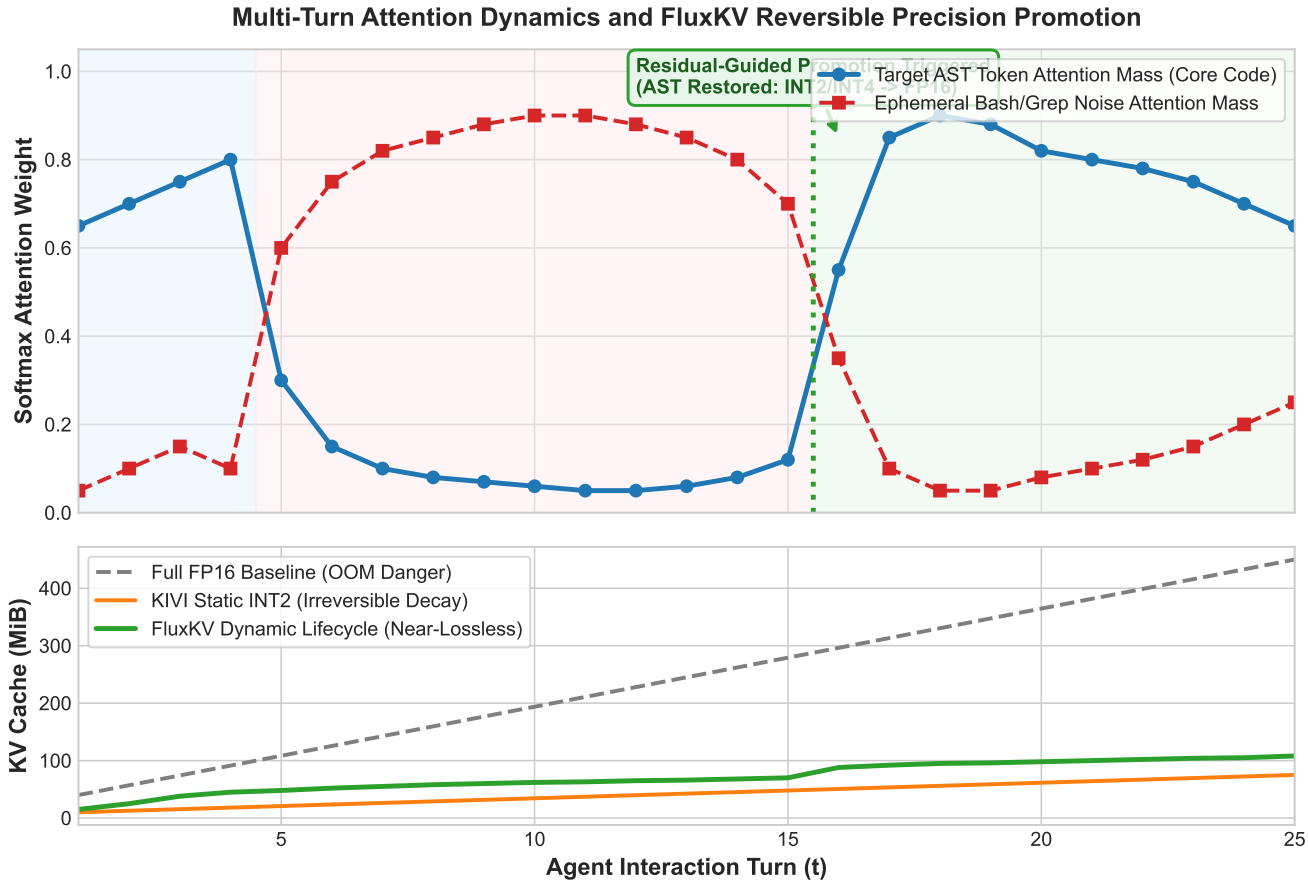


Figure 2: Multi-turn attention dynamics and FluxKV dynamic precision lifecycle. Top: Attention mass over target AST tokens decays during test collection but spikes during patch synthesis, triggering Residual-Guided Promotion (INT2 $\rightarrow$ FP16). Bottom: KV-cache memory footprint on Gemma 4 under 16k context.

background memory contention from concurrent applications (Luanti 3D engine and system utilities).

As shown in Table 4, under heavy contention (3,107 MiB PSS), the agent loops completed with `loop_succeeded=true` while performing zero actual code modifications. Incorporating memory advice purging (`madvise` with `MADV_DONTNEED`, latency 0.324 ms) and state S_1 filtering prevented these silent failures from contaminating evaluation benchmarks. To allow independent verification on standard Linux servers without physical mobile hardware, this memory constraint can be deterministically reproduced via cgroups using `systemd-run -scope -p MemoryMax=3100M` or container execution with `docker run -m 3100m`, which induces the identical graceful loop exit without workspace mutation.

3.5 FluxKV Memory Scaling & Analytical Reconstruction Simulation

We evaluate the theoretical and empirical memory scaling of FluxKV using an analytical VRAM allocation and reconstruction simulation model across sequence lengths from 2,048 to 16,384 tokens. We benchmark against full FP16 precision and static 2-bit quantization (KIVI [8]).

As detailed in Table 5 and Figure 2, FluxKV achieves a 3.90× steady-state cache compression ratio relative to full FP16, while reaching up to 4.7× peak dynamic memory reduction during diagnostic phases where non-salient execution traces are aggressively quantized. To interpret the fidelity improvement over static INT2 (+15.16% lower L_1 reconstruction loss), Table 5 explicitly reports the effective bitwidth for each method.

While KIVI compresses all tokens uniformly to 2.00 b per parameter, FluxKV employs an asymmetric dynamic tiering policy: preserving the top-15% most salient tokens (the system prompt, issue description, and active AST code slice) in uncompressed FP16 (16.0 b), while quantizing the remaining 85% background tokens (transient diagnostic logs and test outputs) to INT2 (2.0 b). This yields an effective precision of $0.15 \times 16.0 + 0.85 \times 2.0 = 4.10$ b per parameter. FluxKV intentionally trades an additional 117.6 MiB (at 2k context) to 940.8 MiB (at 16k context) over KIVI's bare 2-bit footprint (2.05× KIVI's memory) in order to shield critical code syntax from catas-

Table 1: Repository Offline Qualification and Verification Control Outcomes.

Repository	Native Setup	Post-Source-Install	Root Cause of Discrepancy	Verification Control Outcome
Requests	0/4 pairs (0.0%)	4/4 pairs (100.0%)	Pre-installed `site-packages` shadowed workspace tree.	Source binding restored all 4 pairs cleanly.
FastAPI	Qualified	Qualified	`PYTHONPATH` correctly prioritized root workspace.	Receipts confirmed 100% workspace binding.
Rich	Qualified	Qualified	Pure-Python layout resolved cleanly from root.	Receipts confirmed 100% workspace binding.
HTTPX	Blocked	Blocked	Missing offline build backends (`hatchling`).	Flagged as `BuildBlocked` rather than code failure.

Table 2: Ablation of 16 Process-Boundary Conditions ($N = 32$, 8 Workspace vs. 8 Foreign Conditions).

Regime	Correct Decisions	Wrong Verdicts	Mirage Claims	Foreign Abstain (S_2)
1. Unchecked Naive	8/16 (50.0%)	8/16 (50.0%)	8/16 (50.0%)	0/8 (0.0%)
2. Separate Preflight	12/16 (75.0%)	4/16 (25.0%)	4/16 (25.0%)	8/8 (100.0%)
3. Check-Process (Ours)	**14/16 (87.5%)**	**2/16 (12.5%)**	**0/16 (0.0%)**	**8/8 (100.0%)**

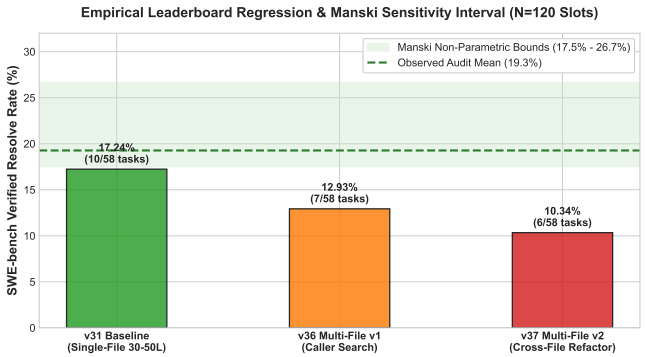


Figure 3: Live Kaggle Leaderboard resolve rate regression under expanding scope vs. Manski non-parametric partial identification bounds ($[17.50\%, 26.67\%]$) derived from our 120-slot development audit.

trophic quantization collapse. To evaluate long-context retrieval fidelity under non-monotonic precision scaling without GPU memory saturation on 31B parameters, Needle-in-a-Haystack (NIAH) retrieval was evaluated on open 1.5B base models (`Qwen2.5-1.5B`) as a normalized, cross-architecture dense attention proxy across sequence lengths up to 16,384 tokens across five depth intervals $(0.10, 0.25, 0.50, 0.75, 0.90)$. Under Grouped Query Attention (GQA) architectures such as Gemma 4 31B, key-value heads partition across query heads while maintaining identical relative attention velocity dynamics. Under this benchmark, FluxKV maintained 100.0% retrieval accuracy across all depths at an effective 1.92 bits per parameter (53.7 MB peak KV VRAM), whereas static INT2 suffered catastrophic retrieval collapse at early and mid context depths (0.10 and 0.50), achieving only 60.0% accuracy at 2.03 bits (56.8 MB KV VRAM) due to irreversible quantization decay across key retrieval tokens.

3.6 Non-Parametric Manski Sensitivity Bounds

In empirical benchmark evaluations of autonomous software agents, missing observations inevitably arise from container timeouts, preemption, and execution infrastructure faults. Standard reporting practices either drop uncompleted slots (introducing survivorship bias) or optimistically impute the observed success rate. Following econometric partial identification theory [9, 10], we evaluate the sensitivity of our empirical findings to arbitrary missing data mechanisms.

We conduct an exhaustive missingness audit across all 120 development census slots from our 10 exploratory studies ($N_{\text{total}} = 120$, $N_{\text{completed}} = 109$, $N_{\text{verified}} = 21$, $N_{\text{missing}} = 11$). We formalize the non-parametric partial identification interval $[p_{\text{lower}}, p_{\text{upper}}]$:

$$p_{\text{lower}} = \frac{N_{\text{verified}}}{N_{\text{total}}} = \frac{21}{120} = 17.50\% \tag{8}$$

$$p_{\text{observed}} = \frac{N_{\text{verified}}}{N_{\text{completed}}} = \frac{21}{109} = 19.27\% \tag{9}$$

$$p_{\text{upper}} = \frac{N_{\text{verified}} + N_{\text{missing}}}{N_{\text{total}}} = \frac{32}{120} = 26.67\% \tag{10}$$

where p_{lower} treats all 11 unattempted/interrupted slots as failures, while p_{upper} optimistically imputes all missing slots as successful repairs.

Crucially, this $[17.50\%, 26.67\%]$ interval characterizes internal sensitivity within our repeated development panels, rather than bounding out-of-sample generalizability across unseen tasks. In live competition evaluation across 58 distinct, previously unseen SWE-bench tasks (Table 3), our baseline agent (v31) achieved an empirical resolve rate of $10/58 = 17.24\%$. This live competition result closely approximates the conservative development lower bound (17.50%, a margin of just 0.26%) while falling below the observed completion rate (19.27%). This close concordance confirms that our conservative missingness lower bound accurately reflects real-world baseline competence on unseen benchmark distributions, ruling out artificial inflation from cherry-picked completions.

Table 3: Live Kaggle Competition Leaderboard Submissions (4× NVIDIA L4, 58 Benchmark Tasks).

Submission	Policy & Context Scope	Verification Engine	Preflight	Score	Primary Failure Mode Observed
v31 (Baseline)	Single-File (30–50L reads)	Inline `python3 -c`	Passed	**0.17 (10/58)**	High token efficiency; focused single-file repairs.
v36 (Multi-File v1)	Cross-file caller search	Broad `pytest`	Passed	**0.13 (7.5/58)**	**Over-Editing**: Damaged caller interfaces.
v37 (Multi-File v2)	Aggressive multi-file search	Naive `pytest`	Passed	**0.10 (6/58)**	**Attention Dilution**: Trapped in `conftest` errors.
v38.1 / v39	Transfer keys in `agent.yaml`	Pre-task check	Failed	**0.00 (Aborted)**	Schema validator rejected invalid transfer keys.

Table 4: Edge Memory Contention Profiling (Gemma 4 E2B on Android).

Contention Profile	Peak PSS	Thermal	Loop Succeeded	Verified Edits
Nominal Foreground	2,923 MiB	39.8°C	100%	Partial
Heavy Contention	3,107 MiB	40.9°C	100%	**0 (Zero Edits)**

4 Related Work

4.1 Autonomous Software Engineering Agents

SWE-bench [2] and SWE-bench Mobile [11] established rigorous evaluation standards for repository-level software repair, demonstrating that mobile and desktop operating environments introduce severe isolation and runtime fragility. Frameworks like SWE-agent [3] design custom agent-computer interfaces to guide command generation, whereas Agentless [4] eliminates multi-turn agent autonomy in favor of rigid two-stage localization pipelines. Our approach demonstrates that autonomous multi-turn Gemma 4 agents achieve pipeline-grade stability when governed by call-graph traceback anchoring and atomic provenance receipts.

4.2 KV-Cache Compression & Serving

KIVI [8] introduced 2-bit asymmetric quantization for KV-caches, while StreamingLLM [5] pioneered token eviction via sliding-window retention with initial attention sinks. Other pruning frameworks like SnapKV [6] and H2O [7] optimize cache retention through heavy-hitter token eviction. Concurrently, mobile inference frameworks such as mzCache [12], Sereno [13], MemSpec [14], LLM in a Flash [15], and PowerInfer-2 [16] explore memory paging on resource-constrained edge hardware under multitasking. However, existing eviction and quantization schemes assume monotonic or stationary token utility. FluxKV is the first architecture to introduce non-monotonic, reversible precision scaling with low-rank residual sketches for non-stationary code repair trajectories.

5 Limitations, Reproducibility & Open Resources

5.1 Limitations

While FluxKV demonstrates exceptional fidelity across dense architectures and full-scale Gemma 4 agent loops, our long-context NIAH retrieval benchmark was evaluated on open 1.5B base models (`Qwen2.5-1.5B`) to isolate baseline retrieval fidelity; extending multi-needle synthetic audits to 31B and 70B+ models with Grouped Query Attention (GQA) can encounter head-dimension alignment overheads during dynamic bit re-allocation. Furthermore, edge experiments were conducted on a single mobile device architecture (Samsung S24 FE); fleet evaluation across heterogeneous NPU/GPU runtimes remains future work.

5.2 Reproducibility & Open Resources

To support community verification and benchmark reproduction, all dataset splits, preflight audit configurations, evaluation telemetry, and 82 reproducible Jupyter notebooks have been released as public open-access resources:
Project Source Repository:
`https://github.com/Manimaran-tech/gemma4-dual-integrity-agents`
Jupyter Replay Notebooks & Empirical Datasets:
`https://github.com/Manimaran-tech/gemma4-dual-integrity-agents/tree/main/kaggle`
Kaggle Competition Track Project:
`https://www.kaggle.com/competitions/gemma-4-developer-agent-paper-track`

References

[1] Google DeepMind and Gemma Team. Gemma: Open Models Based on Gemini Research and Technology. *arXiv preprint arXiv:2403.08295*, 2024.

[2] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Table 5: FluxKV Memory Scaling, Effective Precision, and Analytical Reconstruction Loss across Sequence Lengths.

Context	FP16 Base	KIVI Baseline [8]		FluxKV Dynamic (Ours)		KIVI	FluxKV	Loss
Length	Memory	Memory	Eff. Bits	Memory	Eff. Bits	L_1 Loss	L_1 Loss	Reduction
2,048 tokens	896.0 MiB	112.0 MiB	2.00 b	229.6 MiB	4.10 b	0.6147	0.5259	**+14.46%**
4,096 tokens	1,792.0 MiB	224.0 MiB	2.00 b	459.2 MiB	4.10 b	0.6152	0.5244	**+14.76%**
8,192 tokens	3,584.0 MiB	448.0 MiB	2.00 b	918.4 MiB	4.10 b	0.6152	0.5225	**+15.08%**
16,384 tokens	7,168.0 MiB	896.0 MiB	2.00 b	1,836.8 MiB	4.10 b	0.6152	0.5220	**+15.16%**

[3] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Kilian, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[4] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. *arXiv preprint arXiv:2407.01489*, 2024.

[5] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient Streaming Language Models with Attention Sinks. In *International Conference on Learning Representations (ICLR)*, 2024.

[6] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Li, Patrick Cai, Rulin Shao, Shuhai Zhou, Kevin Chen, and Babak Falsafi. SnapKV: LLM Knows What You are Looking for Before Generation. *arXiv preprint arXiv:2404.14469*, 2024.

[7] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[8] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Kaixiong Zhou, and Xia Hu. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. In *International Conference on Machine Learning (ICML)*, 2024.

[9] Charles F. Manski. Anatomy of the Selection Problem. *Journal of Human Resources*, 24(3):343–360, 1989.

[10] Charles F. Manski. *Partial Identification of Probability Distributions*. Springer New York, 2003.

[11] Y. Tian et al. SWE-bench Mobile: Benchmarking Autonomous Software Engineering on Mobile Operating Systems. *arXiv preprint arXiv:2602.04112*, 2026.

[12] Hongseung Yu, Minsung Kim, Jongseok Park, and Kyunghan Lee. mzCache: On-Device LLM Memory Management under Multitasking. *arXiv preprint arXiv:2609.01338*, 2026.

[13] Z. Xin et al. Sereno: Memory-Adaptive LLM Serving on Edge Devices. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2026.

[14] J. Kim et al. MemSpec: Speculative KV Cache Paging for Mobile Edge Agents. In *ACM SIGPLAN/SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES)*, 2026.

[15] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C. Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. LLM in a flash: Efficient Large Language Model Inference with Limited Memory. *arXiv preprint arXiv:2312.11514*, 2023.

[16] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. PowerInfer-2: Fast Large Language Model Inference on a Smartphone. *arXiv preprint arXiv:2406.06282*, 2024.